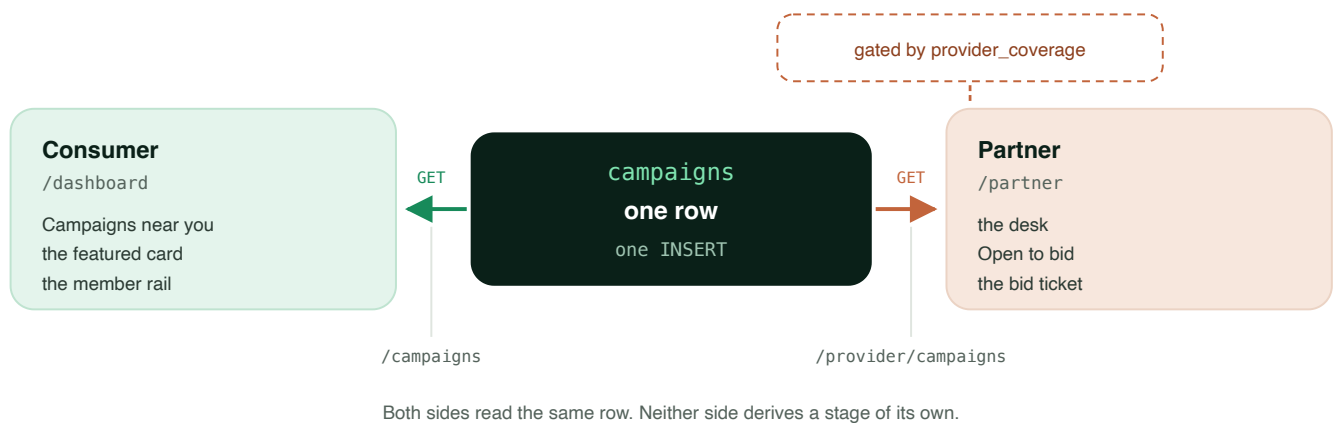


Opening a cohort

One row in one table puts a campaign in front of households **and** in front of partners. This is what that row has to say, in what order to write it, and why the newest cohort is the featured one.



Who this is for

The operator with ZCQL write access in the Catalyst console.

Companion files

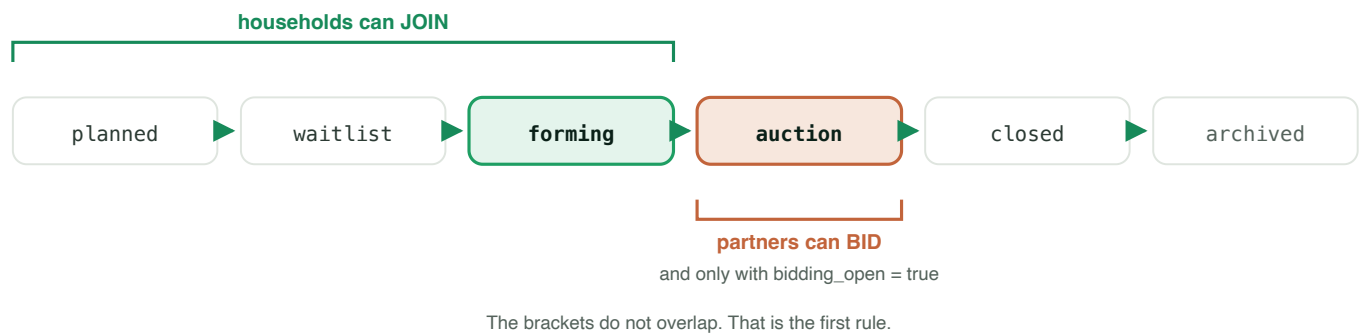
scripts/cohort.mjs,
docs/COHORT_RUNBOOK.md, create-
tables.md sections 16, 18, 20.

Standing rule

The tools print statements. They never write. Every write is yours, pasted deliberately.

Three rules that decide everything else

Nothing in the command list makes sense until these are true in your head. Each one is a single column doing a single job.



Rule 1. A cohort takes joins or bids, never both

`kind` is one column. Joining is open on `planned` , `waitlist` and `forming` . Bidding is open on `auction` . A sealed auction runs against a locked household count, so the moment bidding opens, joining is over, permanently. There is no join path back.

Rule 2. The server owns the stage. The browser only renders it

Stage is derived on every read from the seven calendar dates, never stored and never computed in a browser. A visitor's clock running four minutes fast must not be able to disagree with the server about whether a sealed window is open. Labels come down the wire; countdowns offset from the `serverTime` in the same response.

Rule 3. Featured is not a column. It is the lowest `sort_order`

The catalog is sorted ascending, so the smallest `sort_order` is the first card. To make the newest cohort featured you count **down**, which is also why adding a cohort never has to rewrite the ones already there.

Every column in the row, and what it decides

Eleven columns plus a seven date calendar. Step 3 writes all of them in one statement, so this is the reference for what you are actually saying, not a second thing to type.

COLUMN	WRITTEN AS	WHAT IT DECIDES
campaign_id	slug	The key both dashboards read and every bid receipt is built from. Immutable. Never rendered.
region	text	What a household reads, and the only thing a partner's coverage is matched against.
sub	text	The line under the region on the card, for example "Autumn cohort". Optional.
kind	one of six	Joins, bids, or neither. Must be lowercase: anything else is silently read as <code>planned</code> .

COLUMN	WRITTEN AS	WHAT IT DECIDES
target	number	The households a forming cohort aims at. Left out on an auction, where the count is the joins, not a goal.
seed_members seed_households	number	Added to real joins on both surfaces. Keep at 0 unless you are staging a demo: a seed is padding a partner will bid against.
bidding_open	true / false	The bid window. Only meaningful while <code>kind = 'auction'</code> , and never opened by a date.
sort_order	number	Card order, ascending. The lowest is the featured cohort. See part two.
updated_by updated_at	text, datetime	Who and when. Write <code>'manual'</code> when the change came from ZCQL rather than a route.
the seven dates	datetime, UTC	The calendar, all optional. They set the stage label on both sides, and <code>bidding_closes_at</code> is the only one with teeth: past it, every bid is refused.

Datetimes are UTC, in Catalyst's own format

`'YYYY-MM-DD HH:MM:SS'`, which is neither ISO-8601 nor your local time. The generator prints them correctly. Typing one by hand is how a cohort ends up closing at 5pm in a timezone nobody lives in.

What actually opens a bid window

Four conditions, all four checked on the server on every bid write. Three of them are rows you control. The fourth is the clock.

CONDITION	WHERE IT LIVES	IF IT IS WRONG
<code>kind = 'auction'</code>	campaigns	The desk shows the cohort with no bid button.
<code>bidding_open = true</code>	campaigns	Same. Entering auction never opens bidding on its own, by design.
<code>bidding_closes_at ahead</code>	campaigns	Every bid is refused while the row still reads open. Dates may close a window and may never open one.
<code>bidding_enabled</code>	site_config	Global kill switch. Absent means on.

Two failures that are completely silent

A region no partner can declare. The region name is the entire join between a cohort and a partner, matched by slug, server side. Name a cohort for a place outside the 37 declarable regions and it renders on both dashboards, takes joins, runs its clock down, and takes no bids. Nothing is logged. It reads as a market that did not bite.

A missing table. `campaigns` and `campaign_members` fall back to a code catalog and seed counts when they cannot be read, so their absence looks like a working site. `provider_terms` is the opposite: it fails closed, and while it is missing no bid can be placed at all.

Why the newest cohort is the featured one

You asked for cohort B, created after A, to be featured, with A sliding to position 1. That works, and it takes exactly one number.

BEFORE: one cohort

Cohort A

FEATURED

sort_order 100

position 0

↓

INSERT B, sort_order 99

↓

AFTER: nothing was rewritten

Cohort B

99 · FEATURED

Cohort A

100 · position 1

THE LADDER: start at 100, count down

97	fourth cohort created	featured today
98	third cohort	position 1
99	second cohort	position 2
100	first cohort ever	position 3

The next one takes 96. No UPDATE to any row above it.

The alternative, and why not

Renumbering upward means one UPDATE per existing cohort, every time. The one you miss is the one that stays featured, and nothing tells you.

Ascending sort means lower is earlier. Counting down is what makes "newest first" a single number rather than a migration.

The three things that outrank your number

"Featured" is decided by the member dashboard in this order. Your `sort_order` is the last tiebreak, not the first authority.

ORDER	WHAT WINS	CONSEQUENCE FOR YOU
1st	The member's own cohort	A household that has joined something always sees theirs featured. Nothing you write changes that, and it should not.
2nd	kind, in lifecycle order	A <code>forming</code> cohort sits above every <code>auction</code> one whatever its number. So "newest featured" holds only among cohorts of the same kind . Households read what they can join before what they cannot.
3rd	sort_order, ascending	Your ladder. It orders cohorts inside one kind.

The one thing to accept before you start

A cohort that is open to bid is an `auction`, and an auction is not joinable. So the featured card will read **Sealed bidding** and will carry **no join button**. That is rule 1 doing its job, not a bug.

If the featured card must be joinable, create at `forming` instead, collect the joins, and open the auction afterwards. That is the long path in `docs/COHORT_RUNBOOK.md`. You cannot have both at once on one cohort.

The sequence, start to finish

Seven steps. The **TERMINAL** steps print statements and write nothing. The **ZCQL** steps are what you paste into Catalyst console, Data Store, ZCQL, environment Development.

Fill these in once

PLACEHOLDER	WHAT IT IS	NOT NEGOTIABLE
<ID>	the slug, <code>campaign_id</code>	3 to 64 characters of a-z, 0-9, hyphen. Permanent: both dashboards key on it and every bid receipt is built from it. Never appears on screen.
<REGION>	the region name	Must be one of the 37. This is the whole join to a partner.
<REGION_SLUG>	region, lowercased, runs of non-alphanumerics to one hyphen	Only used to build <code>coverage_key</code> .
<SORT>	<code>sort_order</code>	Lower is featured. Start at 100, take one off per cohort.
<DAYS>	days until bids close	1 to 6 opens the window now. 8 or more writes <code>bidding_open = false</code> .
<ORG_ID>	the partner org that should see it	From <code>SELECT org_id, legal_name FROM provider_orgs;</code>

1

Prove the tables exist **ZCQL**

Each of these must return **without an error**. Zero rows is a pass. Run this once per environment, not once per cohort.

```
SELECT ROWID FROM campaigns LIMIT 1;
SELECT announce_at, bidding_opens_at, bidding_closes_at, offers_at,
       decision_at, switch_window_at, reconcile_at FROM campaigns LIMIT 1;
SELECT ROWID FROM campaign_members LIMIT 1;
SELECT coverage_key, org_id, region, status FROM provider_coverage LIMIT 1;
SELECT acceptance_key, org_id, doc_type, doc_version FROM provider_terms LIMIT 1;
```

Anything that errors gets built from `create-tables.md`, sections 16, 18 and 20. An existence check that needs no console at all: `cd catalyst-backend && catalyst ds:export --table campaigns --page 1`

2

Check the name can be bid on TERMINAL

This is the check ZCQL cannot do for you, and the one that has already produced live cohorts nobody could bid on.

```
node scripts/cohort.mjs regions
```

`<REGION>` must be in that list. `<ID>` is usually its slug but does not have to be: `region` is what renders and what coverage matches, `<ID>` is only the key.

3

Create it, open to bid, in one statement TERMINAL ZCQL

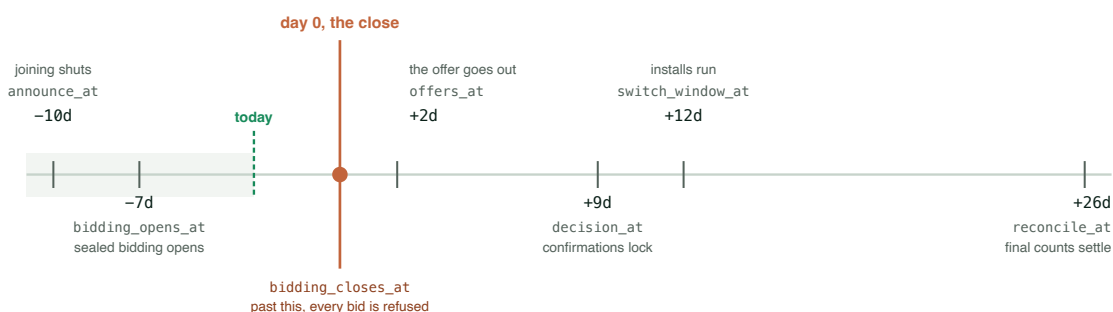
One `INSERT` is the entire creation, for both dashboards at once. There is no second row to write for the consumer side.

```
node scripts/cohort.mjs seed <ID> --regions "<REGION>" --first <DAYS> --sort <SORT>
```

Paste the one `INSERT` it prints. It carries `kind = 'auction'`, `bidding_open = true` and all seven calendar dates:

```
INSERT INTO campaigns (campaign_id, region, kind, seed_members, seed_households,
  bidding_open, sort_order, updated_by, updated_at, announce_at, bidding_opens_at,
  bidding_closes_at, offers_at, decision_at, switch_window_at, reconcile_at)
VALUES ('<ID>', '<REGION>', 'auction', 0, 0, true, <SORT>, 'manual', '<now>',
/* the seven dates, spaced around the close: */
  '<close -10d>', '<close -7d>', '<close>', '<close +2d>',
  '<close +9d>', '<close +12d>', '<close +26d>');
```

`--regions` can be dropped when `<ID>` title-cases into `<REGION>` exactly. Seeds stay at 0 deliberately: a seed count is added to real joins on both surfaces, so a non-zero seed is padding a partner will eventually bid against.



Deadlines land at 21:00 UTC, every other date at 13:00 UTC, which is 9 AM Eastern.

The offsets are days from the close, and nothing else. `<DAYS>` only decides where day 0 lands. Set it small and the two negative dates are already behind, which is exactly what writes `bidding_open = true`. Set it to 8 and `bidding_opens_at` is still ahead, so the row is written closed and the desk reads Announced.

3b

The same thing on a test rail, in minutes

TERMINAL

ZCQL

For a test case rather than a launch. `--minutes` replaces `--first` and dates the whole batch from now, so one statement lands a cohort that is **featured on the consumer side and biddable on the desk** straight away.

```
node scripts/cohort.mjs seed <ID> --minutes 5 --sort <SORT>
```

COLUMN	LANDS AT	WHY THERE
announce_at	-2 intervals	joining has already shut
bidding_opens_at	-1 interval	behind you deliberately. The desk draws its button on stage open or closing, and neither is reachable until this date has passed
bidding_closes_at	+1 interval	so the bid window is two intervals wide : ten minutes at <code>--minutes 5</code> , twenty at 10
offers_at	+2	the rest runs one stage per interval
decision_at	+3	and this one is read first by the stage engine, so it ends the run
switch_window_at	+4	
reconcile_at	+5	the whole rail finishes here, 25 minutes out at <code>--minutes 5</code>

2 is the floor, and 1 is refused rather than quietly unwatchable: the catalog is memoized for 60 seconds. Every cohort in a batch runs the same rail, so a row of cards stays comparable. `--minutes` and `--first` cannot be combined. And note the member rail only advances on its own for a member who has **joined** that cohort, which an auction cannot be, so a full rail test means creating at `forming`, joining, then moving to auction.

4

Give it coverage, or it reaches nobody

TERMINAL

ZCQL

A `campaigns` row alone is on **no** partner's desk. It reaches every partner whose declared coverage matches the region and is active, and nobody else. One row per org, per region.

```
node scripts/cohort.mjs coverage <ORG_ID> --region "<REGION>"
```

That prints the `INSERT`, and the `UPDATE` to use when the row already exists. New coverage lands `verifying`, which greys the desk row with "Verifies with <REGION> coverage". Move it on:

```
UPDATE provider_coverage
SET status = 'active', verified_at = '<now>', rejection_reason = '', updated_at =
'<now>'
WHERE coverage_key = '<ORG_ID>:<REGION_SLUG>';
```

If that errors on the last two columns they do not exist yet. Drop them and set `status` alone, which is the fallback the admin route takes.

5

Clear the three gates on the bid itself

ZCQL

Only needed if a partner is meant to actually place a bid, not just see the cohort. Each of these refuses with its own message, so the message tells you which one.

```
UPDATE provider_orgs SET approval_status = 'approved' WHERE org_id = '<ORG_ID>';
UPDATE provider_users SET role = 'admin' WHERE user_id = '<USER_ID>';
SELECT config_key, value FROM site_config WHERE config_key = 'bidding_enabled';
```

Orgs are created `pending` and nothing self-approves. The first seat in an org is `admin` and every seat after is `viewer`, which cannot bid. `bidding_enabled` absent means on. The standard cohort terms are accepted by the partner in their own console, so `provider_terms` needs to exist but needs no row from you.

6

Every later cohort takes a lower number

TERMINAL

ZCQL

This is the featured flow. Cohort A is already in the table at `<SORT_A>`. The new one gets `<SORT_A> - 1`.

```
node scripts/cohort.mjs seed <ID_B> --regions "<REGION_B>" --first <DAYS> --sort <SORT_B>
```

B is now featured, A is position 1, both are live, both are open to bid, and the A row was never touched. Then step 4 again for `<REGION_B>`: coverage is per region, so a partner covering A's region sees nothing of B's.

To hand the featured slot back, move the newer cohort **below** the older one rather than moving the older one up, so only one row is ever rewritten:

```
UPDATE campaigns SET sort_order = <SORT_A + 1> WHERE campaign_id = '<ID_B>';
```

7

Verify what both sides will read

TERMINAL

ZCQL

Wait 60 seconds first. The catalog is memoized for a minute, and hand-written ZCQL cannot invalidate it the way the admin routes do.

```
node scripts/cohort.mjs verify <ID>
```

```
SELECT campaign_id, region, kind, bidding_open, sort_order FROM campaigns;  
SELECT org_id, region, status FROM provider_coverage;
```

That first result, read top to bottom, is the card order on `/dashboard` inside each `kind` bucket, which groups 6f and 6g of `scripts/qa-dashboard.mjs` assert in a browser. Then, signed in on each side: `GET /api/auth/campaigns` and `GET /api/auth/provider/campaigns`. Check `live` on both. `live: false` means `campaign_members` was unreadable and every count is a seed, which looks exactly like a cohort nobody joined.

Two habits worth keeping

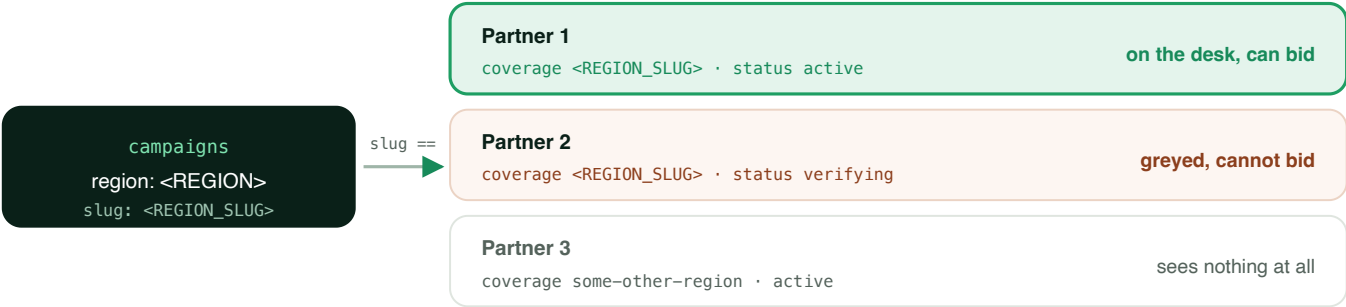
Never DELETE a campaign row. There are no foreign keys, so its `campaign_members` rows survive pointing at nothing. A duplicate key error means the cohort exists: `SELECT` it and `UPDATE` what you meant to change.

Let the tool write the literals. ZCQL here has no parameter binding, so `cohort.mjs` refuses a value containing a quote or a backslash rather than escaping it. A rejected value is a typo you fix. An escaped one is a habit, in a console with full write access.

What opens a window, and what fails silently

Read this page before you conclude that something is broken. The single most common surprise is a cohort that is in the table, visible to households, and absent from every partner desk.

Coverage: who a cohort reaches



Partner 3 is not an error state. No partner ever learns a cohort outside their coverage exists.

One minute after the INSERT

	CONSUMER, /DASHBOARD	PARTNER, /PARTNER
Appears as	a card in "Campaigns near you", badge Sealed bidding	a desk row, only where coverage is active
Stage	bidding	open , or closing in the last 24 hours
Can act	No. An auction is not joinable	Yes, once steps 4 and 5 are done
Refreshes	on its own, every 15 seconds while the calendar fits in 36 hours	never. The console fetches once at boot, so reload it after any ZCQL write

The first row you ever insert hides the other six

An **empty** campaigns table falls back to the six-region code catalog. A populated one is the truth. So the moment your first cohort lands, the six demo regions disappear from both dashboards. Insert every region you want visible, or expect a one-card site.

When it still looks wrong

WHAT YOU SEE	WHAT IT IS
Nothing changed on either dashboard	The 60 second catalog memo. Wait, then reload.
The new cohort is not featured, and kinds differ	Working as designed. kind is read before sort_order .
The new cohort is not featured, all are auctions	Its sort_order is not the lowest. Check step 7's first query.
The featured card has no join button	It is an auction. Auctions are never joinable.
One member sees a different featured card	They have joined a cohort. Theirs always wins.

WHAT YOU SEE

WHAT IT IS

The row is missing entirely, and `kind` looks right

`kind` is not lowercase, or not one of the six. Anything else is read as `planned` and nothing is logged.

Desk reads **Announced**, no bid button

`bidding_opens_at` is still ahead, so `bidding_open` was written `false`. Use a smaller `<DAYS>`, or `cohort.mjs bidding <ID> --on`.

`bidding_open` is true and every bid is refused

`bidding_closes_at` has passed, or `bidding_enabled` is false.

"Bidding is not available right now"

`bid_revisions` or the fifteen extra `provider_bids` columns do not exist. Section 18.

`bidding_open` reads false right after you set it

The bare `true` literal was not accepted. Try `'true'`, then `1`, then the row's checkbox in the Data Store UI. All three are read as true.

Counts frozen, `live: false`

`campaign_members` is unreadable. Every count on screen is a seed.